

数论-板子+笔记

一、 位运算

基本的位运算：共 6 种，分别为按位与、按位或、按位异或、按位取反、左移和右移。

Tips: 注意区分 逻辑与 ($\wedge$) 和按位与 ($\&$)、逻辑或 ($\vee$) 和按位或 ($\mid$) 的区别。

异或：

$a \wedge a = 0, a \wedge 0 = a$

交换 a, b : $a = a \wedge b, b = a \wedge b, a = a \wedge b$

快速寻找数组中出现奇数次的数 (其他都为偶数)

a 是奇数 $a \wedge 1 = a - 1$, 因为消除零 // a 是偶数 $a \wedge 1 = a + 1$

$x \wedge (1 \ll i)$ 把 x 的第 i 位翻转

$a \text{ xor } (a \text{ xor } b) = (a \text{ xor } a) \text{ xor } b = 0 \text{ xor } b = b$

取反：

是对一个数 num 进行的位运算，即单目运算。其对应的运算符为 $\sim$ 。

它的作用是把 num 的二进制补码中的 0 和 1 全部取反 (0 变为 1, 1 变为 0)。有符号整数的符号位在 $\sim$ 运算中同样会取反。

原码：最高位表示符号：0 为正，1 为负，其余位表示数值的绝对值。

补码：在二进制表示下，正数和 0 的补码为其本身，负数的补码是将其对应正数按位取反后+1。

****反码：**正数：反码 = 原码 负数：符号位保持为 1，其余位按位取反 (0 $\rightarrow$ 1, 1 $\rightarrow$ 0)。

****左移：** $num \ll i$ 表示将 num 的二进制表示向左移动 i 位所得的值。

右移： $num \gg i$ 表示将 num 的二进制表示向右移动 i 位所得的值。

![[Pasted image 20260317152839.png]]

题面：输入 n ，输出小于 n 的最大的 2 的整数次幂

```
int n;
cin >> n;
int ans = 1 << 31 - __builtin_clz(n);
cout << ans << '\n';
```

二、 快速幂与快速乘整除性与约数

快速幂模板：

```
int fastpow(int x, int a)
{
    int ans = 1;
    while(a){
        if (a&1) ans *= x;
        x *= x;
        a >>= 1;
    }
    return ans;
}
```

快速乘模板：

```
int Mul(int a, int b){
    int ans = 0;
    while(b){
        if(b&1) ans += a;
        a+=a;
        b >>= 1;
    }
    return ans;
}
```

简化阶乘求法：

```
#include <stdio.h>
#include <math.h>

double funcos( double e, double x );

int main() {
    double e, x;
    scanf("%lf %lf", &e, &x);
    printf("cos(%.2f) = %.6f\n", x, funcos(e, x));
    return 0;
}

/* 你的代码将被嵌在这里 */

double funcos(double e, double x) {
    double sum = 1.0;        // 第0项: 1
    double term = 1.0;       // 当前项
    int n = 1;
    do{
        // 计算新项:  $(-1)^n * x^{(2n)} / (2n)!$ 
        term = term * (-1) * x * x / ((2*n-1) * (2*n));
        sum += term;
        n++;
    } while(fabs(term) >= e);
    return sum;
}
```

三、素数理论

素数的分布：开数组时空间除以10

埃及筛：

```
const int MAXN = 1e6 + 5;
bool prim[MAXN];

void sieve(int n){
    fill(prim + 2, prim + n + 1, true);
    for (int i = 2; i * i <= n; i++){
        if (prim[i]){
            for (int j = i * i; j <= n; j += i)
                prim[j] = false;
        }
    }
}
```

线性筛（欧拉筛）：埃式筛（未去重）的升级版

原理：使用最小质因数去标记每个合数

```
const int N = 1e7;
vector<int> prim;
vector<int> p(N);
void primesa(){
    for(int i=2; i<N; ++i){
        if(p[i] == 0){
            p[i] = i;
            prim.push_back(i);
        }
        for(auto j : prim){
            if(i * j >= N) break;
            p[i*j] = j;
            if(i % j == 0) break;
            //意味着如果继续往下筛，`i * primes[j+1]` 的最小质因子将会是 `primes[j]` 而不是 `primes[j+1]`，这会导致重复标记。
        }
    }
}
```

区间筛：

场景：求区间 $[L,R]$ 内的素数，其中 R 很大（如 10^{12} ），但区间长度 $R-L$ 较小（如 10^6 ）。

核心思想： $[L,R]$ 内的合数必有一个不超过 $\sqrt{R}$ 的质因子。

1. 先预处理出 $1 \sim \sqrt{R}$ 内的所有素数。
2. 用这些素数去筛掉区间 $[L,R]$ 内的倍数。

```

typedef long long ll;
void segmented_sieve(ll L, ll R) {
    vector<bool> is_p(R - L + 1, true);
    if (L == 1) is_p[0] = false;

    for (ll i = 2; i * i <= R; i++) {
        if (isPrime(i)) { // 使用预处理的素数
            // 找到大于等于L的第一个i的倍数
            for (ll j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
                is_p[j - L] = false;
        }
    }
}

```

四、同余运算与剩余系

扩展中国剩余定理 (EXCRT)

应用场景：求解一元线性同余方程组（即韩信点兵问题）。

$$x \equiv a_1 \pmod{m_1} x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2} x \equiv a_2 \pmod{m_2} \dots$$

前提条件（极苛刻）：所有的模数 $m_1, m_2, \dots, m_n$ 必须两两互质！

它的思想是通过 ExGCD，把两个方程合并成一个方程，极其精妙：

```

// 扩展中国剩余定理 (EXCRT)
// 解决模数 m[i] 不互质的情况！
// 返回值：方程组的最小非负整数解，如果无解返回 -1
ll EXCRT(int n, ll a[], ll m[]) {
    ll ans = a[1], M = m[1]; // 初始化：先满足第一个方程

    for (int i = 2; i <= n; i++) {
        // 当前要解的合并方程：ans + k * M ≡ a[i] (mod m[i])
        // 转化为 ExGCD 形式：k * M + y * m[i] = a[i] - ans
        ll A = M, B = m[i], C = (a[i] - ans % B + B) % B; // C 是常数项，防止负数
        ll x, y;
        ll d = exgcd(A, B, x, y);

        // 如果 C 不能被 gcd(A, B) 整除，根据裴蜀定理，方程无解！
        if (C % d != 0) return -1;

        // 算出一组特解 x
        ll bg = B / d; // 模数缩放
        x = (ll)((__int128)x * (C / d) % bg);
        x = (x % bg + bg) % bg; // 保证 x 是正数

        // 更新总答案和总模数
        ans += x * M;
        M *= bg; // 更新总模数为 lcm(M, m[i])
        ans = (ans % M + M) % M;
    }
    return ans;
}

```

裴蜀定理

```
#include<bits/stdc++.h>
using namespace std;

void exgcd(int a, int b, int &x, int &y){
    if(b==0){
        x=1, y=0;
        return;
    }
    exgcd(b, a&b, x, y);
    int t = x;
    x = y;
    y = t-a/b*y;
}

void work(){
    int n;
    cin >> n;
    vector<int> a(n);
    int last = 0;
    for(int i=0; i<n; i++){
        cin >> a[i];
        if(a[i] < 0) a[i] *= (-1);
        last = gcd(last, a[i]);
    }
    cout << (last<0 ? -last : last) << '\n';
}

signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    int T=1;
    // cin>>T;
    while(T-->0) work();
}
```

五、乘法逆元

定义：如果 $a \cdot x \equiv 1 \pmod m$ ，我们就称 x 是 a 在模 m 下的乘法逆元，记作 a^{-1} 。

作用：在模 m 意义下，除以一个数等于乘以这个数的逆元。

$$a/b \pmod m = (a \cdot b^{-1}) \pmod m$$

三个板子：

方法 A：费马小定理（最常用）

- 前提条件： m 必须是质数，且 a 不是 m 的倍数。
- 原理： $a^{m-2} \equiv a^{-1} \pmod m$ 。
- 实现：使用快速幂。

```
typedef long long ll;
```

```

11 fast_pow(11 a, 11 b, 11 m) {
    11 res = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}

// 求 a 在模 m 下的逆元
11 inv(11 a, 11 m) {
    return fast_pow(a, m - 2, m);
}

```

方法 B: 扩展欧几里得算法 (exGCD)

- 前提条件: a 与 m 互质 ($\gcd(a,m)=1$) , m 不一定是质数。
- 原理: 解方程 $ax+my=1$, 求出的 x 即为逆元。

```

11 exgcd(11 a, 11 b, 11 &x, 11 &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    11 d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
}

11 inv_exgcd(11 a, 11 m) {
    11 x, y;
    11 d = exgcd(a, m, x, y);
    if (d != 1) return -1; // 无逆元
    return (x % m + m) % m; // 确保结果为正数
}

```

方法 C: 线性求逆元 (求 1~n 所有数的逆元)

- 场景: 当需要频繁用到 1~N 范围内的所有逆元 (如组合数计算) 。
- 时间复杂度: $O(N)$ 。

```

11 inv[MAXN];
void get_all_inv(int n, int m) {
    inv[1] = 1;
    for (int i = 2; i <= n; i++) {
        // 公式: inv[i] = (m - m/i) * inv[m%i] % m
        inv[i] = (long long)(m - m / i) * inv[m % i] % m;
    }
}

```

应用：带模除法与逆元

在计算 $ab \pmod m$ 时，步骤如下：

1. 计算 b 的逆元 b^{-1} 。
2. 结果为 $(a * b_inv) \% m$ 。

```
int exp(int a, int b){
    int res = 1;
    a = a%mod;
    while(b){
        if(b%2 == 1) res=(a*res) % mod;
        a *= a;
        a = a%mod;
        b /= 2;
    }
    return res % mod;
}

int inv(int b){
    return exp(b,mod-2) % mod;
}
```

注意点：

- aa 本身可能是很大计算出来的结果，记得先对 aa 取模： $(a \% m * b_inv) \% m$ 。
- 只有当 $\gcd(b,m)=1$ 时，逆元才存在。

应用：组合数取模板子：\$Cnk(modp)\$

```
const int MOD = 1e9 + 7;
const int MAXN = 100005;
ll fact[MAXN], invFact[MAXN];

ll fast_pow(ll a, ll b) { /* 见上文 */ }

// 预处理阶乘和阶乘的逆元
void init() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;

    // 阶乘逆元的推导：(n!)的逆元 = ((n+1)!)的逆元 * (n+1)
    invFact[MAXN - 1] = fast_pow(fact[MAXN - 1], MOD - 2);
    for (int i = MAXN - 2; i >= 0; i--) {
        invFact[i] = invFact[i + 1] * (i + 1) % MOD;
    }
}

ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    // 公式: n! / (r! * (n-r)!)
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}
```

六、 欧拉函数与欧拉定理

七、 离散对数

辗转相除法求gcd

```
long long llgcd(int a,int b) {  
    if(b==0) return a;  
    return llgcd(b,a%b);  
}
```

通过gcd求最小公倍数

```
long long lcm(long long a, long long b) {  
    if (a == 0 || b == 0) return 0;  
    long long g = llgcd(a, b);  
    return llabs((a / g) * b); //先除后乘，减少溢出风险  
}
```

求两个数的最大公共质因数

// 欧几里得算法

```
long long gcd_ll(long long a, long long b) {  
    while (b){  
        long long r = a % b;  
        a = b;  
        b = r;  
    }  
    return a >= 0 ? a : -a;  
}  
  
// n 的最大质因数 (n<=1 返回 n, 本题里 g=0/1 需要特判)  
long long max_prime_factor(long long n) {  
    if (n <= 1) return n;  
    long long ans = -1;  
    while (n % 2 == 0){  
        ans = 2;  
        n /= 2;  
    }  
    for (long long d = 3; d * d <= n; d += 2){  
        while (n % d == 0){  
            ans = d;  
            n /= d;  
        }  
    }  
    if (n > 1) ans = n;  
    return ans;  
}
```


